МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего профессионального образования

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Инженерная школа информационных технологий и робототехники

Направление 15.04.06 «Мехатроника и робототехника»

Отделение автоматизации и робототехники

Построение нейросетевого классификатора для идентификации типа подстилающей поверхности при движении мобильного робота в условиях неоднородной среды

Отчет по лабораторной работе № 3

по дисциплине «Методы искусственного интеллекта в мехатронике и робототехнике»

Выполнил ст. гр. 8ЕМ51	_____	_____	Якушев Н.Е.
Проверил старший преподаватель ОАР ИШИТР	_____	_____	Андраханов А.А.
Должность	Подпись	Дата	Фамилия И.О.

Томск 2026

Оглавление

Цель работы....3
Задачи....3
Вариант задания....3
Ход работы.... 4
1 Загрузка и обработка данных выборки “Data_Set_(A+B).xlsx”.... 4
1.1 Общее описание всего процесса обработки данных.... 4
1.2 Первичный анализ исходного датасета.... 4
1.3 Анализ датасета с добавленными подмножествами.... 5
1.4 Проведение сортировки обработанного датасета.... 9
2 Проведение исследований с моделью MLPClassifier в зависимости от способа предобработки данных.... 11
2.1 Общее описание этапов исследования.... 11
2.2 Проведение исследований с подбором гиперпараметров для данных без дополнительной обработки.... 14
2.3 Проведение исследований с подбором гиперпараметров для данных с сортировкой относительно целевого класса.... 18
2.4 Проведение исследований с подбором гиперпараметров для данных с нормализацией параметров методом MinMaxScaler.... 21
2.5 Проведение исследований с подбором гиперпараметров для данных с нормализацией параметров и балансировкой SMOTE.... 24
2.6 Проведение исследований с подбором гиперпараметров для данных с нормализацией параметров и балансировкой ADASYN.... 27
2.7 Проведение исследований с использованием Optuna.... 30
2.8 Подведение итогов исследований.... 34
3 Тестирование модели на тестовой выборке C.... 36
3.1 Загрузка и обработка новой выборки данных.... 36
3.2 Оценка качества ранее найденных моделей.... 38
Вывод.... 41
Список используемых источников.... 43
Приложение А (обязательное) Графики параметров выборки AB датасета.... 44
Приложение Б (обязательное) Графики параметров выборки C.... 45

Цель работы

- Приобретение практических навыков в решении одной из актуальных задач в построении интеллектуальной СУ автономной робототехнической системы, функционирующей в условиях физически неоднородной среды.

- Практическое изучение особенностей и способов решения задачи несбалансированной классификации, а также расширение инструментария разработчика в части работы с методами машинного обучения.

- Развитие исследовательских навыков работы с методом машинного обучения на примере прикладной задачи и реальной выборке данных.

- Развитие навыков программирования систем вычислительного интеллекта на Python.

Задачи

1. Загрузка и подготовка данных из файла "*Data_Set_(A+B).xlsx*".
2. Без дополнительной обработки данных провести исследования по подбору гиперпараметров для модели *MLPClassifier*.
3. Провести исследование по подбору гиперпараметров для модели *MLPClassifier* ***с отсортированными данными***.
4. Провести исследование по подбору гиперпараметров для модели *MLPClassifier* ***с нормализованными данными через MinMaxScaler***.
5. Провести исследование по подбору гиперпараметров для модели *MLPClassifier* ***с нормализованными данными и применением методов балансировки целевого класса с помощью SMOTE и ADASYN.***
6. На основе лучших результатов обучения всех моделей и обработки данных провести проверку на ***выборке C из файла "Data_Set_C.xlsx"***.
7. Подвести итоги исследований.

Вариант задания

Вариант типа поверхности: **5** (вариант 20).

Вариант датасета: {***V1; V3***} (вариант 20).

Ход работы

1 Загрузка и обработка данных выборки “*Data_Set_(A+B).xlsx*”

1.1 Общее описание всего процесса обработки данных

Обработка данных состоит из следующих этапов:

1. Исследование исходного датасета на выбросы, *NaN*-ячейки и дублирующиеся строки.
2. Добавление в исходный датасет множества $\{V2\}$ и $\{V3\}$.
3. Отрисовка всех столбцов и проведение исследований по данным (нахождение динамически идентичных параметров, определение динамики и выбросов).
4. Удаление лишних строк и обработка выбросов.
5. Сохранение обработанного датасета “*Data_Set_AB_processed.xlsx*”.
6. Проведение сортировки данных для распределения целевого класса между параметрами.
7. Сохранение датасета с сортировкой в файл “*Data_Set_sorted.xlsx*”.

1.2 Первичный анализ исходного датасета

Первичный анализ исходного датасета включал использование методов *.info()*, *.describe()* и *.duplicated*() из библиотеки “*Pandas*”. Для проведения анализа. Были сделаны следующие выводы:

1. Отсутствие *NaN*-ячеек данных и дубликатов (все данные уникальны).
2. Датасет содержит 176 строк с информацией о данных датчиков для 5 типов поверхностей, что является небольшим объемом выборки.
3. В датасете наблюдается дисбаланс классов при условии идентификации одного типа поверхности (в среднем 36 строк для одной поверхности).

Результаты анализа показали отсутствие необходимости в предварительной очистке данных, поэтому было принято решение о расчёте и добавлении подмножеств $\{V2\}$ и $\{V3\}$.

1.3 Анализ датасета с добавленными подмножествами

Подмножество $\{V1\} = \{\{N_1, N_2, N_3\}, \{\omega_1, \omega_2, \omega_3\}, \{I_1, I_2, I_3\}, \{g_x, g_y, g_z\}, \{a_x, a_y, a_z\}\}$ является исходным и содержится в файле “*Data_Set_(A+B).xlsx*”.

Для подробного анализа датасета был произведет расчёт подмножеств $\{V2\}$ и $\{V3\}$, после чего они были добавлены в исходный датасет.

Подмножество $\{V2\} = \{V_x, V_y, \Omega, I_x, I_y, I_\varphi, I_\Sigma\}$ рассчитывается на основе формул, приведенных на рисунке 1.

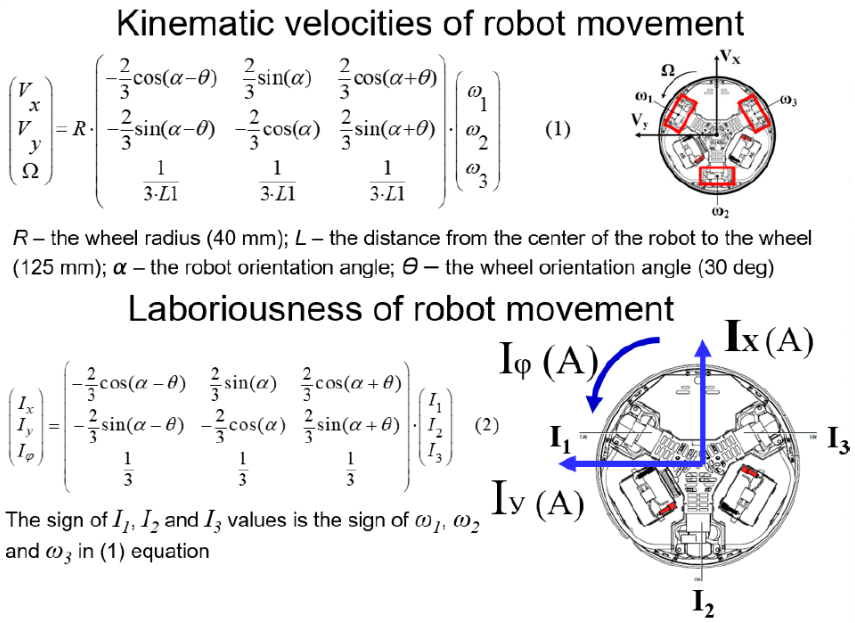


Рисунок 1 – Кинематические преобразования для получения интегральных величин, характеризующих скорость и трудоемкость перемещения робота по поверхности

Подмножество $\{V3\} = \{T_x, T_y, T_\varphi, T_z\}$ состоит из относительных параметров, рассчитанных из параметров подмножества $\{V2\}$ по формулам:

$$T_x = \frac{V_x}{I_x}, T_y = \frac{V_y}{I_y}, T_\varphi = \frac{\Omega}{I_\varphi}, T_z = \frac{g_z}{I_\varphi}.$$

Для анализа полного датасета были выполнены следующие операции:

1. Применение метода *.describe()* позволило определить выбросы и масштабы рассчитанных параметров.

2. Проведение обработки выбросов с использованием метода *кэпирования по перцентилям*.

3. Отрисовка всех графиков с наложением типов поверхностей для определения визуальных признаков для целевой поверхности.

4. Построение матрицы корреляции для определения линейной зависимости между параметрами.

По итогам анализа были сделаны следующие выводы:

1. Показания энкодеров [N_1, N_2, N_3] практически эквивалентны данным [w_1, w_2, w_3].

2. Наблюдается динамическое совпадение показаний гироскопа по оси Z и рассчитанной угловой скорости Ω.

3. Показания датчиков тока [I_1, I_2, I_3], гироскопа и акселерометра уникальны и практически отфильтрованы.

4. У параметров T_z и T_φ имеется единичный выброс (скорее всего из-за того, что I_φ близка к нулю). Ранее уже была произведена обработка выброса для корректного анализа корреляции переменной с параметрами.

5. На основе матрицы корреляции можно отметить, что многие параметры сильно коррелируют между собой, хотя и не имеют прямой физической связи (например, g_y и g_z).

На рисунке 2 представлена матрица корреляции до обработки данных. На рисунке 3 представлен график параметров, которые динамически схожи друг с другом. В приложении А представлены графики всех параметров для полной выборки данных.

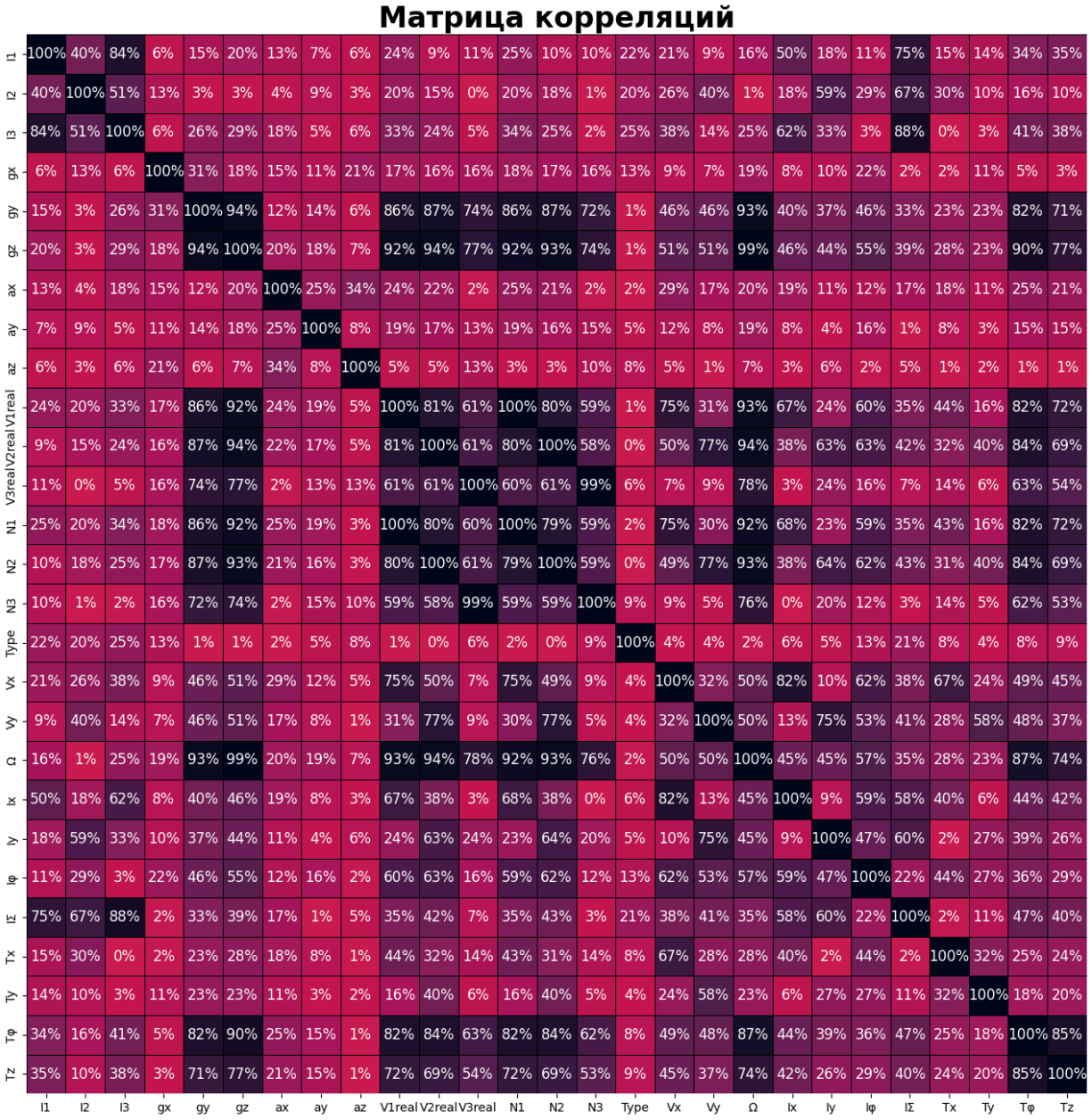


Рисунок 2 – Матрица корреляции для всех данных

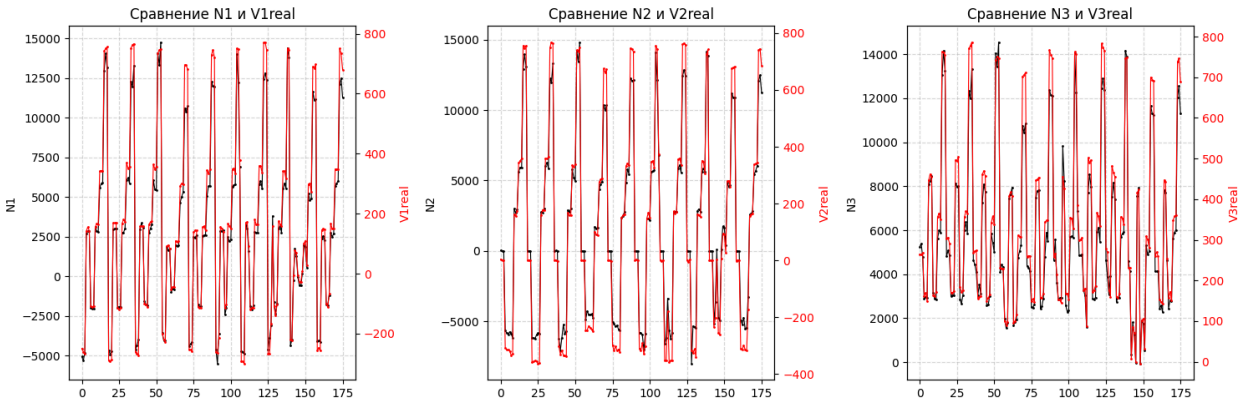


Рисунок 3 – Графики параметров [N_1, N_2, N_3] и [w_1, w_2, w_3]

На основе проведенного анализа для обработки полного датасета были предприняты следующие шаги:

1. Удаление показаний энкодеров [N_1, N_2, N_3] (тип *int64*), так как угловые скорости колёс [w_1, w_2, w_3] имеют тип *float*, что предположительно даст большую точность..

2. При добавлении других выборок данных (“*Data_Set_C.xlsx*” и “*Data_Set_D.xlsx*”) будет производиться обработка возможных выбросов.

3. Удаление всех столбцов подмножества {*V2*}, согласно варианту задания.

4. Присвоение строкам столбца “*Type*” с целевой поверхностью значения “1”, а остальным – “0”.

В дальнейшем обработанный файл был сохранён под названием “*Data_Set_AB_processed.xlsx*”. На рисунке 4 представлена матрица корреляции после обработки данных.

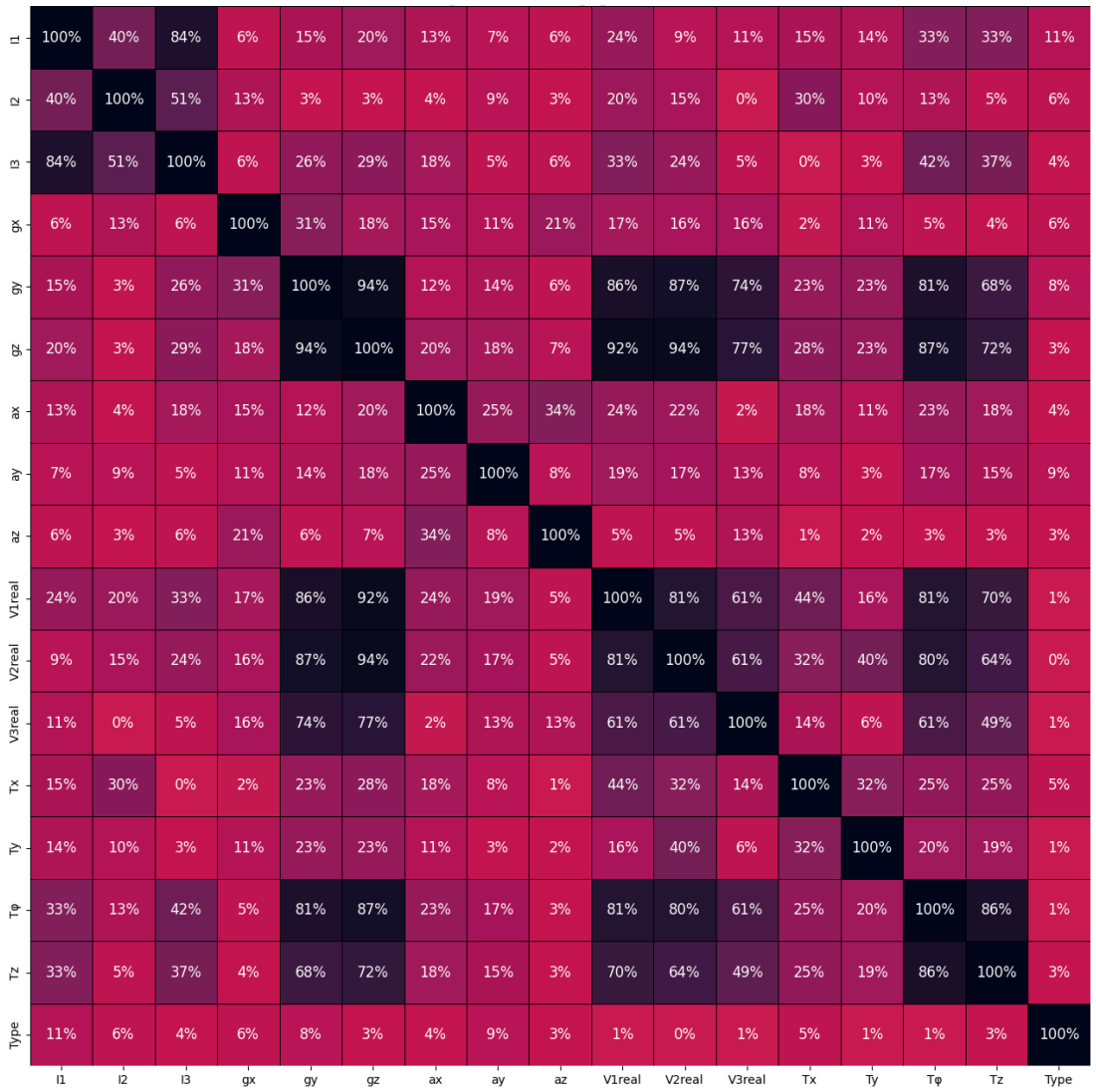


Рисунок 4 – Матрица корреляции для всех данных после обработки

1.4 Проведение сортировки обработанного датасета

Сортировка данных была произведена по принципу равномерного распределения целевого параметра по всему датасету. Для этого была написана функция, которая распределяет целевой параметр в соотношении 4:1, где 1 соответствует целевому классу. Данное соотношение было найдено на основе первичного анализа данных.

На рисунке 5 представлено распределение целевого параметра до и после применения сортировки:

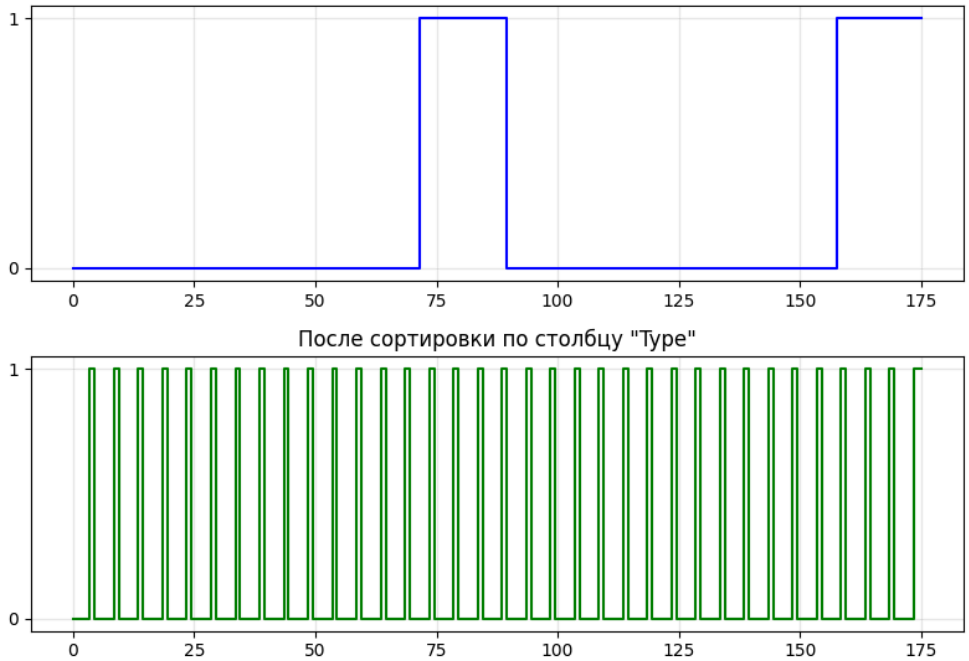


Рисунок 5 – Результат сортировки данных

В дальнейшем обработанный файл был сохранён под названием “*Data_Set_AB_sorted.xlsx*”.

2 Проведение исследований с моделью *MLPClassifier* в зависимости от способа предобработки данных

2.1 Общее описание этапов исследования

Выполнение исследований подразумевает следующие входные условия:

1. В качестве модели используется *MLPClassifier*, которая по своей сути является полносвязной нейронной сетью.
2. Исследуются пять вариантов предобработки данных: без дополнительной обработки, после сортировки (равномерное распределение целевого класса), после нормализации (*MinMaxScaler*), а также после нормализации с балансировкой классов методами *SMOTE* и *ADASYN*.
3. Предлагается осуществить подбор гиперпараметров “*hidden_layer_sizes*”, “*activation*”, “*solver*” и “*max_iter*” для каждого варианта обработки данных.
4. Выбор наилучшей модели осуществляется на основе *F*-меры (*F1-score*).

MLPClassifier [1] – алгоритм многослойного перцептрона (MLP), который обучается по методу обратного распространения ошибки. Модель содержит входной слой (размерность равна числу признаков), один или несколько скрытых полносвязных слоев и выходной слой.

На основе документации и практических рекомендаций можно выделить следующие гиперпараметры:

1. Алгоритм оптимизации: {‘lbfgs’, ‘sgd’, ‘adam’}.

- ‘*lbfgs*’ хорошо работает на малых выборках и потенциально может обеспечить высокие значения *F*-меры.

- ‘*sgd*’ требует ручной настройки ‘*alpha*’, ‘*learning_rate*’ и других параметров (согласно информации с интернета). Для нашего исследования не подходит.

- ‘*adam*’ обладает адаптивным масштабированием градиентов, автоматически подбирая скорость обучения для каждого веса, что делает его устойчивым к выбору начального шага и позволяет обучаться без тонкой настройки.

2. Функция активации: {‘identity’, ‘logistic’, ‘tanh’, ‘relu’}.

- ‘*identity*’ (линейная) обычно применяется в линейных моделях. Согласно матрицы корреляции данная функция активации нам не подходит.

- ‘*logistic*’ склонна к насыщению: при значениях, близких к 0 или 1, градиенты стремятся к нулю, что может остановить обучение. Для многослойных сетей и малых выборок не рекомендуется.

- ‘*tanh*’ сходится быстрее логистической, но проблема насыщения сохраняется.

- ‘*relu*’ сходится быстро, подходит для небольших выборок, более устойчива в многослойных моделях. Теоретически это выглядит как наиболее предпочтительный выбор.

3. Максимальное число итераций (max_iter). Слишком маленькое значение может привести к недообучению, слишком большое – к переобучению. Универсального правила выбора нет.

4. Архитектура скрытых слоёв (hidden_layer_sizes) – основной фактор, влияющий на сложность модели.

- Чем больше слоёв и нейронов, тем выше риск переобучения, особенно на малых выборках.

- При малых выборках не рекомендуется большое количество слоев и нейронов, модель с большей вероятностью переобучится.

- В нашем случае 16 входных параметров даже для одного скрытого слоя дают несколько десятков или сотен настраиваемых связей при 176 наблюдениях.

Теперь необходимо разобраться, каким образом может помочь каждая из сортировок и в чём заключается их особенности:

- *Обучение модели на сортированных данных*. Сортировка равномерно распределяет целевой класс по выборке. Если при обучении данные не используется стратифицированная кросс-валидация с перемешиванием, то данная операция позволит добиться более высокой точности на модели с алгоритмом оптимизации “*lbfgs*”

- *Обучение модели на нормированных данных*. Нормализация позволяет ускорить сходимость решения и устраняет доминирование признаков с большими абсолютными значениями. По теории должен сильно увеличить точность модели, так как все данные имеют разный масштаб.

- *Обучение модели на нормированных данных с использованием SMOTE*. *SMOTE* синтезирует новые примеры целевого класса путём интерполяции между соседями, балансируя выборку и снижая склонность модели игнорировать редкий класс. По теории при балансировке выборки данных появляется возможность выявить больше признаков, по которым будет производится предсказания класса поверхности, что сильно может повысить точность.

- *Обучение модели на нормированных данных с использованием ADASYN*. *ADASYN* адаптивно создаёт больше синтетических примеров в областях, где классификация целевого класса затруднена (на границе решений). По теории, как и со SMOTE, должна повысить точность, но стоит учесть, что параметры имеют мало данных на границе, что может привести к ложному примеру.

2.2 Проведение исследований с подбором гиперпараметров для данных без дополнительной обработки

Для автоматизации экспериментов были разработаны три вспомогательные функции, используемые во всех последующих разделах:

- *train_models(...)*. Необходима для обучения модели по всем гиперпараметрам с сохранением каждого результата в таблицу. Содержит в себе стратифицированную кросс-валидацию (метод "*StratifiedKFold*" из библиотеки "*sklearn.model_selection*"). В отличие от GridSearchCV, функция позволяет управлять выводом и форматом сохраняемых данных.
- *plot_results(...)*. Строит гистограмму для каждой из моделей по Accuracy, F-мере, а также учёт стандартных отклонений (характеризует устойчивость модели). Позволяет наглядно на одном графике проанализировать результаты.
- *plot_model_by_combo_id(...)*. Для конкретной комбинации гиперпараметров выводит график оценки точности модели на всей выборке данных.

Кросс-валидация [2] – это метод оценки модели, при котором данные разбиваются на несколько равных частей (фолдов), на которых уже последовательно обучается модель.

В отличие от однократного разбиения на обучающую и тестовую выборки (метод "*train_test_split*"), кросс-валидация использует все данные и для обучения, и для проверки. Это имеет критическое значение при малом объёме данных, где каждая строка может содержать ценную информацию о процессе.

В данной работе применяется стратифицированная кросс-валидация (метод "*StratifiedKFold*") с четырьмя фолдами и перемешиванием, которая сохраняет пропорции классов в каждой части и исключает влияние порядка строк на результат.

На основе изучения литературы и методических указаний было решено протестировать архитектуры с одним, двумя и тремя скрытыми слоями.

Гипотеза – из-за малого размера выборки и разного масштаба признаков модели продемонстрируют невысокую точность (60–80 %) и *F*-меру (40-60%) и недостаточную устойчивость.

Варианты гиперпараметров приведены в листинге 1.

Листинг 1 – Варианты гиперпараметров для разного количества скрытых слоёв

```
1.  # для одного слоя
2.  hidden_opts = [(10,), (25,), (50,), (100,)]
3.  act_opts = ['relu', 'tanh']
4.  solv_opts = ['adam', 'lbfgs']
5.  iter_opts = [10, 100, 1000]
6.
7.  # для двух слоев
8.  hidden_opts = [(5, 5), (20, 10), (50, 20), (100, 50)]
9.  act_opts = ['relu', 'tanh']
10. solv_opts = ['adam', 'lbfgs']
11. iter_opts = [10, 100, 1000]
12.
13. # для трех слоев
14. hidden_opts = [ (10, 10, 10), (30, 30, 20), (32, 32, 16), (48, 48, 32), (64, 64, 32)]
15. act_opts = ['relu', 'tanh']
16. solv_opts = ['adam', 'lbfgs']
17. iter_opts = [100, 1000]
```

После проведения экспериментов были получены следующие результаты:

- Лучший результат модели с одним слоем: *hidden_layer_sizes*=(50,), *activation*='*relu*', *solver*='*lbfgs*', *max_iter*=1000. ***Accuracy = 0.8182 ± 0.0000, F1-score = 0.5250 ± 0.0433.***

- Лучший результат модели с двумя слоями: *hidden_layer_sizes*=(50, 20), *activation*='relu', *solver*='*lbfgs*', *max_iter*=1000. ***Accuracy = 0.8295 ± 0.0254, F1-score = 0.6055 ± 0.0134.***

- Лучший результат модели с тремя слоями: *hidden_layer_sizes*=(32, 32, 16), *activation*='*relu*', *solver*='*adam*', *max_iter*=1000. ***Accuracy = 0.8523 ± 0.0254, F1-score = 0.5917 ± 0.0759.***

На данном этапе лучший результат показала **двухслойная нейронная сеть** (см. рис. 6). В целом, ожидания, построенные на основе первичного анализа модели и гиперпараметров, совпали с результатами обучения.

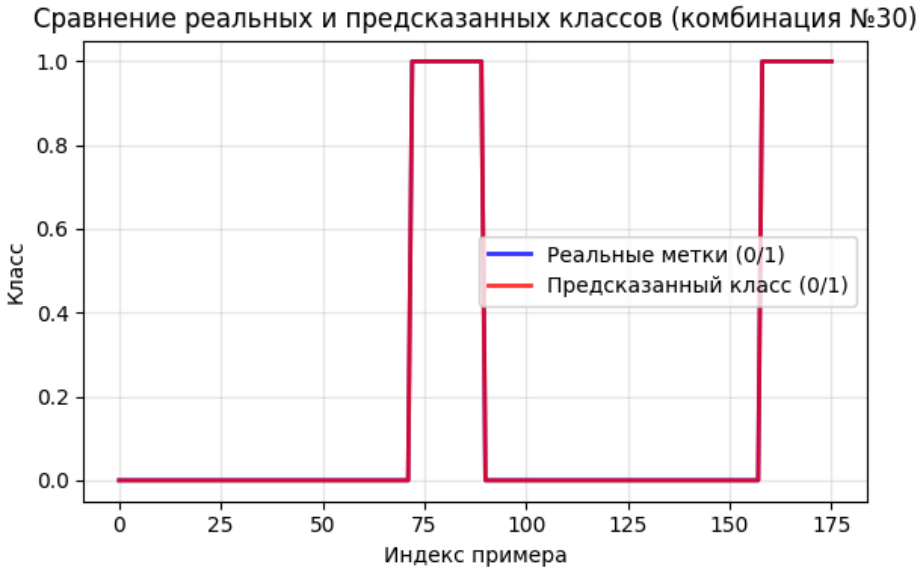


Рисунок 6 – График сравнения табличного значения выходной величины и выхода многосвязной сети для двухслойной модели

Определение лучшей модели происходило на основе построенных гистограмм для всех моделей. Пример гистограммы для двухслойной модели представлен на рисунке 7.

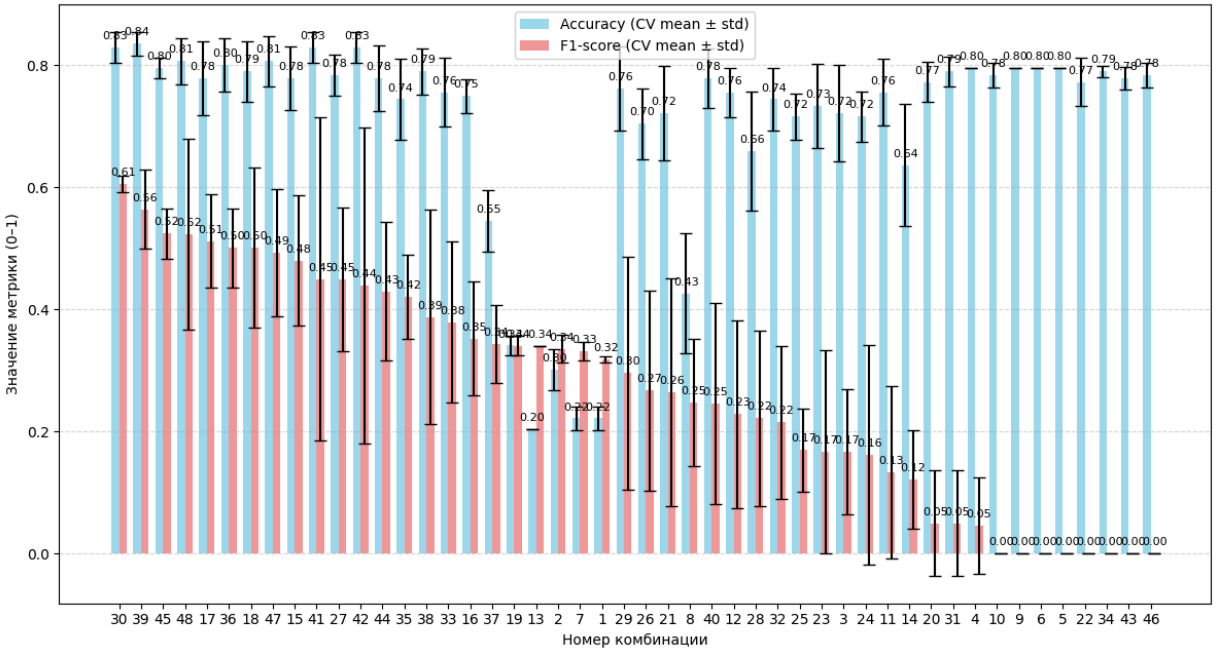


Рисунок 7 – Гистограмма с метриками для каждой комбинации гиперпараметров для двухслойной модели

По результатам исследования можно сделать следующие выводы:

- Вопреки первоначальной гипотезе, модели без нормализации продемонстрировали достаточно высокую точность и устойчивость даже при разбиении на 4 фолда.
- Малые значения ‘*max_iter*’ приводили к недообучению и низкому качеству.
- Функция активации ‘*relu*’ в большинстве запусков обеспечивала наилучший результат, что и предполагалось.
- Сложно выделить паттерны для подбора наилучших гиперпараметров, при этом не уйти в переобучение (увеличение количества нейронов в большинстве повышает точность и *F*-меру, но при этом значительно повышает риск переобучения).
- Лучшая пара на основе результатов обучения – ‘*relu*’ + ‘*lbfgs*’
- Алгоритмы оптимизации ‘*lbfgs*’ и ‘*adam*’ дают сопоставимые результаты.

2.3 Проведение исследований с подбором гиперпараметров для данных с сортировкой относительно целевого класса

Проведение исследований с сортировкой целевого класса по предположениям должен дать более высокий результат за счёт равномерного распределения целевого класса, что может быть крайне полезно для алгоритмов оптимизации (особенно для '*lbfgs*'). Однако в наших экспериментах применяется стратифицированная кросс-валидация "*StratifiedKFold*", которая сама гарантирует равномерное представительство классов в каждом фолде и нивелирует влияние порядка строк. Таким образом, дополнительная сортировка не должна дать значимого выигрыша.

Гипотеза – результаты будут практически идентичны результатам обучения без сортировки.

Варианты гиперпараметров с сортированными входными данными приведены в листинге 2.

Листинг 2 – Варианты гиперпараметров для данных с сортировкой

```
# для одного слоя
hidden_opts = [(25,), (50,), (100,), (200,)]
act_opts = ['relu', 'tanh']
solv_opts = ['adam', 'lbfgs']
iter_opts = [100, 1000, 2000]

# для двух слоев
hidden_opts = [(20, 10), (20, 20), (40, 40), (50, 20), (100, 50)]
act_opts = ['relu', 'tanh']
solv_opts = ['adam', 'lbfgs']
iter_opts = [100, 1000]


# для трех слоев
hidden_opts = [ (10, 10, 10), (30, 30, 20), (32, 32, 16), (48, 48, 32), (64, 64, 32)]
act_opts = ['relu', 'tanh']
solv_opts = ['adam', 'lbfgs']
iter_opts = [100, 1000]
```

После проведения экспериментов были получены следующие результаты:

- Лучший результат модели с одним слоем: *hidden_layer_sizes*=(50,), *activation*='*relu*', *solver*='*lbfgs*', *max_iter*=1000. ***Accuracy = 0.8182 ± 0.0000, F1-score = 0.5250 ± 0.0433.***

- Лучший результат модели с двумя слоями: *hidden_layer_sizes*=(50, 20), *activation*='*relu*', *solver*='*adam*', *max_iter*=1000. ***Accuracy = 0.8409 ± 0.0161, F1-score = 0.5893 ± 0.0246.***

- Лучший результат модели с тремя слоями: *hidden_layer_sizes*=(64, 64, 32), *activation*='*relu*', *solver*='*lbfgs*', *max_iter*=1000. ***Accuracy = 0.8409 ± 0.0161, F1-score = 0.6213 ± 0.0406.***

Наивысшую среднюю F-меру показала **трёхслойная сеть**, однако устойчивость модели ниже, чем у двухслойной. На рисунках 8 и 9 представлены график прогнозов лучшей трехслойной модели и гистограмма метрик для всех трехслойных конфигураций.

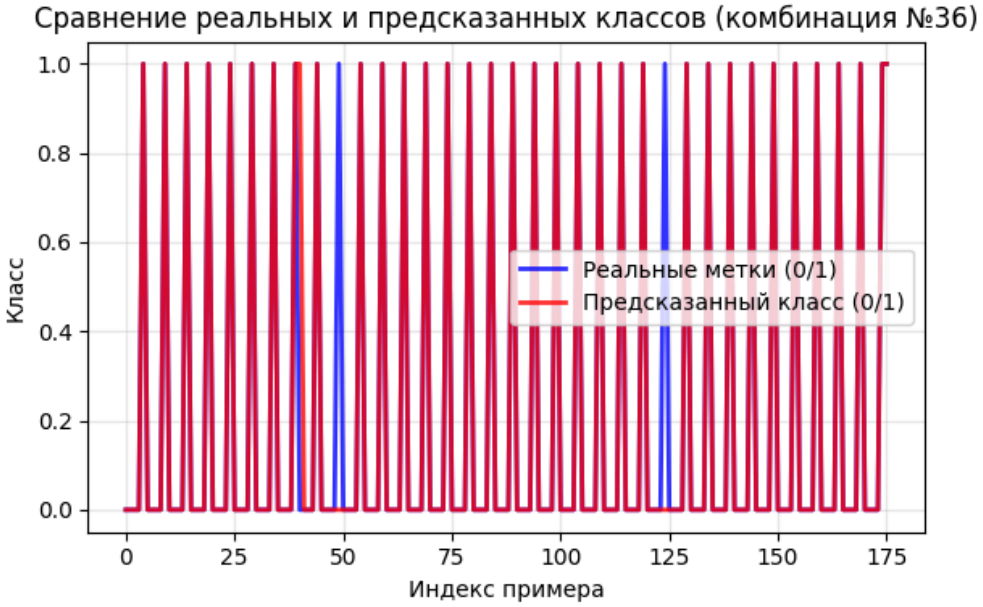


Рисунок 8 – График сравнения табличного значения выходной величины и выхода многосвязной сети для трехслойной модели с сортировкой данных

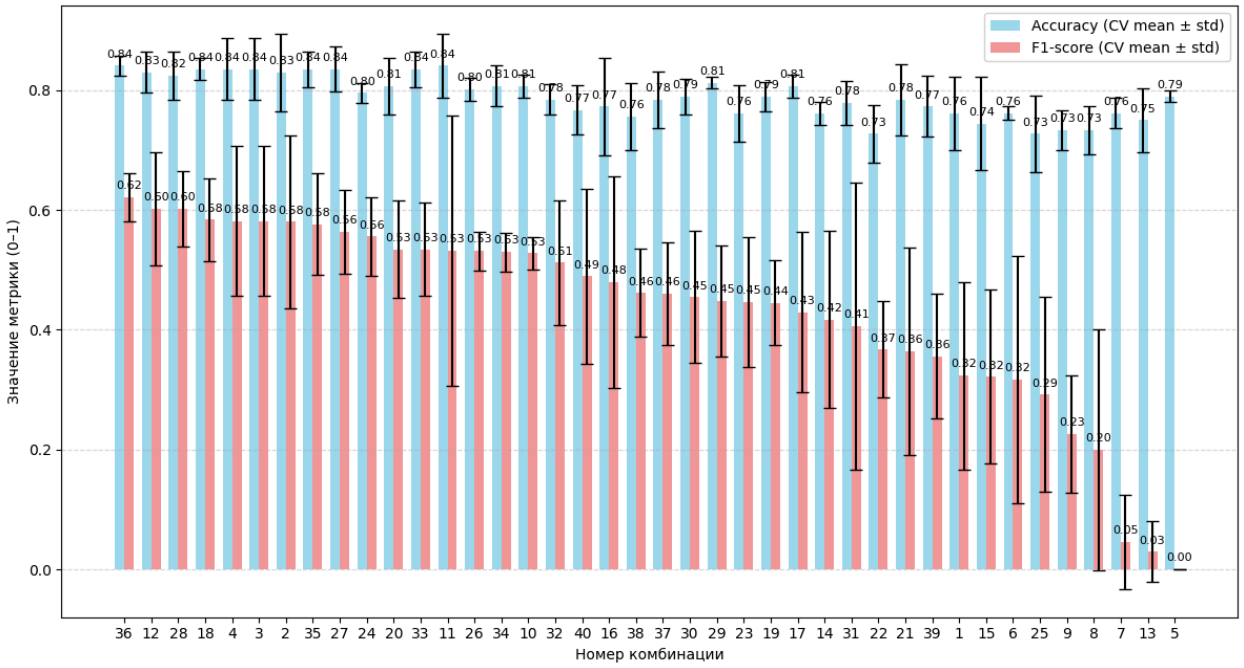


Рисунок 9 – Гистограмма с метриками для каждой комбинации гиперпараметров для трехслойной модели с сортировкой данных

По результатам исследования можно сделать следующие выводы:

- Гипотеза подтвердилась частично, сортировка не дала улучшения показателей. Однако, набор гиперпараметров, обеспечивающий лучшие показатели по *F*-мере и устойчивости изменились, причём конкретной зависимости выделить не получилось.

- Функция активации ‘*relu*’, в сравнении с предыдущим, поднялась выше по рейтингу.

- Лучшая пара на основе результатов обучения – ‘*relu*’ + ‘*lbfgs*’.

- Алгоритмы оптимизации ‘*lbfgs*’ и ‘*adam*’ дают сопоставимые результаты.

2.4 Проведение исследований с подбором гиперпараметров для данных с нормализацией параметров методом MinMaxScaler

Нормализация приводит все признаки к единому диапазону, что важно для градиентных алгоритмов оптимизации, чувствительных к масштабу входных переменных. При разных масштабах одни входы могут подавлять другие в процессе вычисления градиентов, замедляя сходимость и снижая устойчивость обучения. Таким образом, нормализация должна обеспечить более быстрое и стабильное обучение.

Гипотеза – после нормализации точность и *F*-мера моделей повысятся, а их устойчивость улучшится.

Варианты гиперпараметров для данных с нормализацией приведены в листинге 3.

Листинг 3 – Варианты гиперпараметров для данных с нормализацией

```
1.  # для одного слоя
2.  hidden_opts = [(50,), (100,), (150,), (200,)]
3.  act_opts = ['relu', 'tanh']
4.  solv_opts = ['adam', 'lbfgs']
5.  iter_opts = [500, 1000]
6.
7.  # для двух слоев
8.  hidden_opts = [(20, 10), (20, 20), (40, 30), (50, 20), (100, 50)]
9.  act_opts = ['relu', 'tanh']
10. solv_opts = ['adam', 'lbfgs']
11. iter_opts = [500, 1000]
12.
13. # для трех слоев
14. hidden_opts = [ (10, 10, 10), (30, 30, 20), (32, 32, 16), (48, 48, 32), (64, 64, 32)]
15. act_opts = ['relu', 'tanh']
16. solv_opts = ['adam', 'lbfgs']
17. iter_opts = [100, 1000]
```

После проведения экспериментов были получены следующие результаты:

- Лучший результат модели с одним слоем: *hidden_layer_sizes*=(50,), *activation*='*tanh*', solver='*lbfgs*', *max_iter*=500. ***Accuracy = 0.8409 ± 0.0161, F1-score = 0.6307 ± 0.0280.***

- Лучший результат модели с двумя слоями: *hidden_layer_sizes*=(50, 20), *activation*='*relu*', *solver*='*adam*', max_iter=1000. ***Accuracy = 0.8466 ± 0.0188, F1-score = 0.6412 ± 0.0278.***

- Лучший результат модели с тремя слоями: *hidden_layer_sizes*=(30, 30, 20), *activation*='*relu*', *solver*='*adam*', *max_iter*=1000. ***Accuracy = 0.8693 ± 0.0188, F1-score = 0.6671 ± 0.0389.***

На данном этапе лучший результат показала **трехслойная нейронная сеть** (см. рис. 10-11).

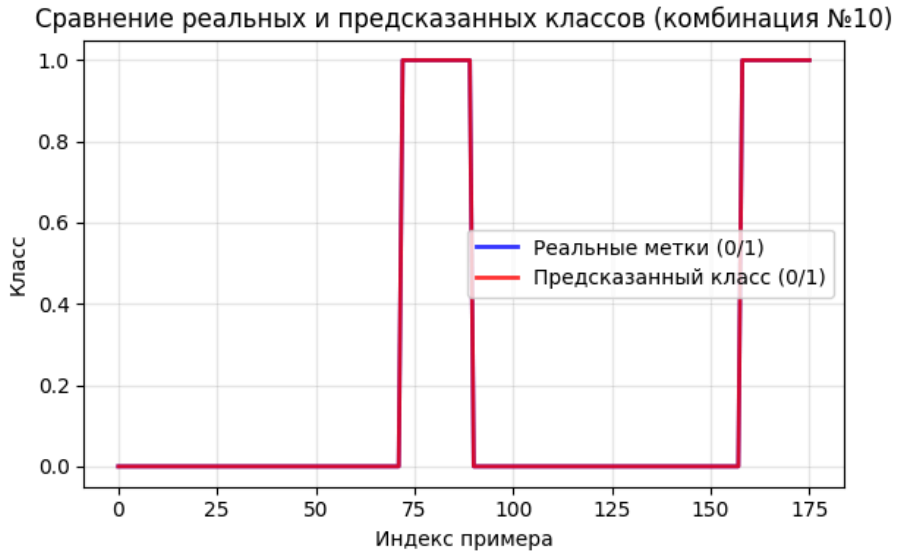


Рисунок 10 – График сравнения табличного значения выходной величины и выхода многосвязной сети для трехслойной модели с нормализацией данных

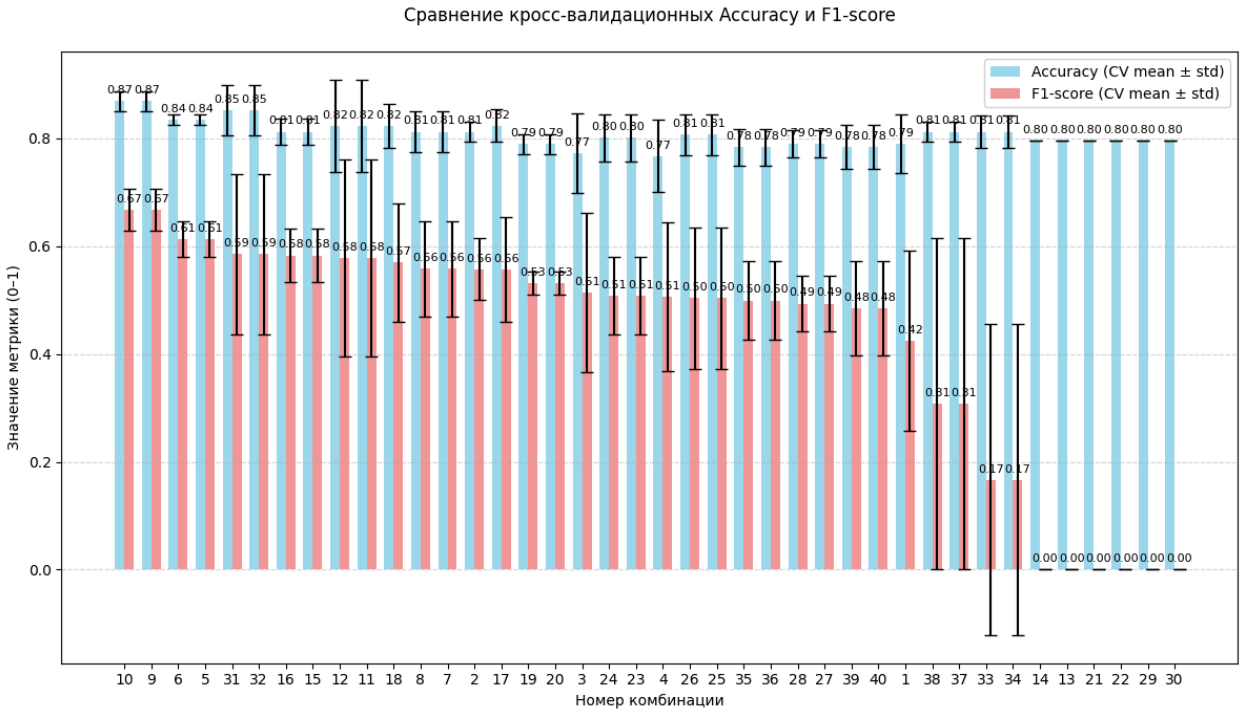


Рисунок 11 – Гистограмма с метриками для каждой комбинации гиперпараметров для трехслойной модели с нормализацией данных

По результатам исследования можно сделать следующие выводы:

- Гипотеза подтвердилась частично. Точность возросла незначительно, однако *F*-мера и устойчивость улучшились существенно по сравнению с ненормализованными данными.

- На однослойной модели, что весьма интересно, ‘*adam*’ и ‘*relu*’ показали результаты значительно хуже. При этом в других моделях изменений в логике практически нет.

- Лучшая пара на основе результатов обучения для двухслойной и трёхслойной модели – ‘*relu*’ + ‘*adam*’.

- Алгоритм ‘*adam*’ стал доминирующим в лучших конфигурациях, не включая однослойную сеть, что подтверждает его эффективность при работе с нормализованными данными.

2.5 Проведение исследований с подбором гиперпараметров для данных с нормализацией параметров и балансировкой SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) [3] синтезирует новые примеры целевого класса путём интерполяции между ближайшими соседями. Это позволяет уравнять количество наблюдений редкого и основного классов, предоставляя классификатору больше информации о целевом классе. Балансировка выполняется после нормализации, так как синтетические точки строятся в пространстве признаков и критически зависят от их масштаба.

Гипотеза – благодаря расширению выборки целевого класса точность, *F*-мера и устойчивость моделей значительно возрастут по сравнению с предыдущим этапом (нормализация без *SMOTE*).

Варианты гиперпараметров для данных с учетом нормализации данных и балансировки *SMOTE* представлен в листинге 4.

Листинг 4 – Варианты гиперпараметров для данных с нормализацией и *SMOTE*

```
1.  # для одного слоя
2.  hidden_opts = [(50,), (100,), (150,), (200,)]
3.  act_opts = ['relu', 'tanh']
4.  solv_opts = ['adam', 'lbfgs']
5.  iter_opts = [500, 1000]
6.
7.  # для двух слоев
8.  hidden_opts = [(20, 10), (20, 20), (40, 30), (50, 20), (100, 50)]
9.  act_opts = ['relu', 'tanh']
10. solv_opts = ['adam', 'lbfgs']
11. iter_opts = [500, 1000]
12.
13. # для трех слоев
14. hidden_opts = [ (10, 10, 10), (30, 30, 20), (32, 32, 16), (48, 48, 32), (64, 64, 32)]
15. act_opts = ['relu', 'tanh']
16. solv_opts = ['adam', 'lbfgs']
17. iter_opts = [100, 1000]
```

После проведения экспериментов были получены следующие результаты:

- Лучший результат модели с одним слоем: *hidden_layer_sizes*=(150,), *activation*='*tanh*', *solver*='*lbfgs*', *max_iter*=1000. ***Accuracy = 0.9536 ± 0.0118, F1-score = 0.9558 ± 0.0109.***

- Лучший результат модели с двумя слоями: *hidden_layer_sizes*=(40, 30), *activation*='*relu*', *solver*='*lbfgs*', *max_iter*=1000. ***Accuracy = 0.9357 ± 0.0071, F1-score = 0.9370 ± 0.0075.***

- Лучший результат модели с тремя слоями: *hidden_layer_sizes*=(48, 48, 32), *activation*='*relu*', *solver*='*lbfgs*', *max_iter*=1000. ***Accuracy = 0.9393 ± 0.0186, F1-score = 0.9426 ± 0.0171.***

На данном этапе лучший результат показала **однослойная нейронная сеть** (см. рис. 12-13).

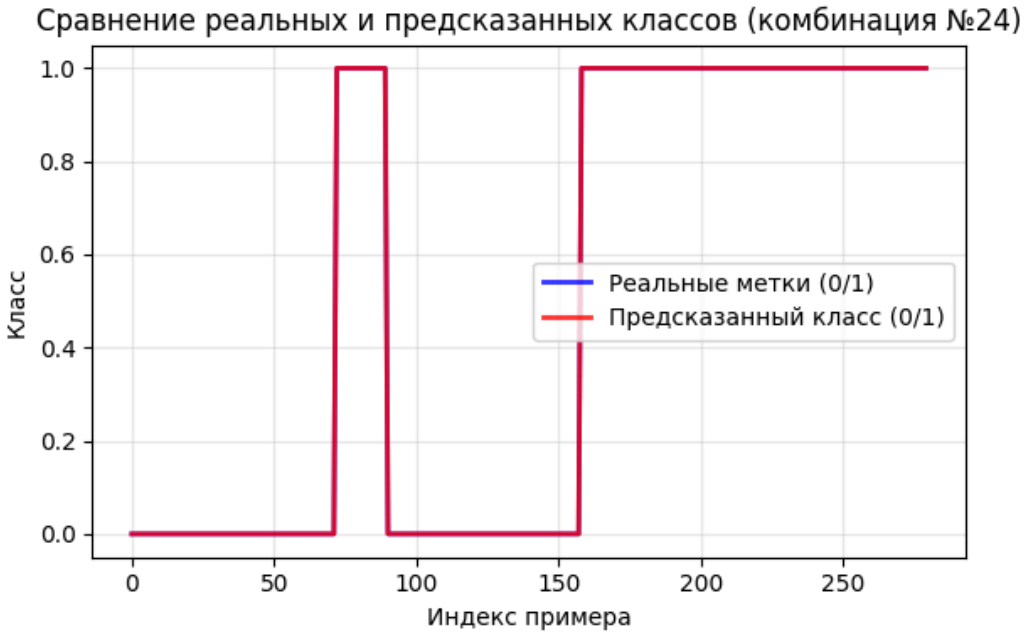


Рисунок 12 – График сравнения табличного значения выходной величины и выхода многосвязной сети для трехслойной модели с балансировкой *SMOTE*

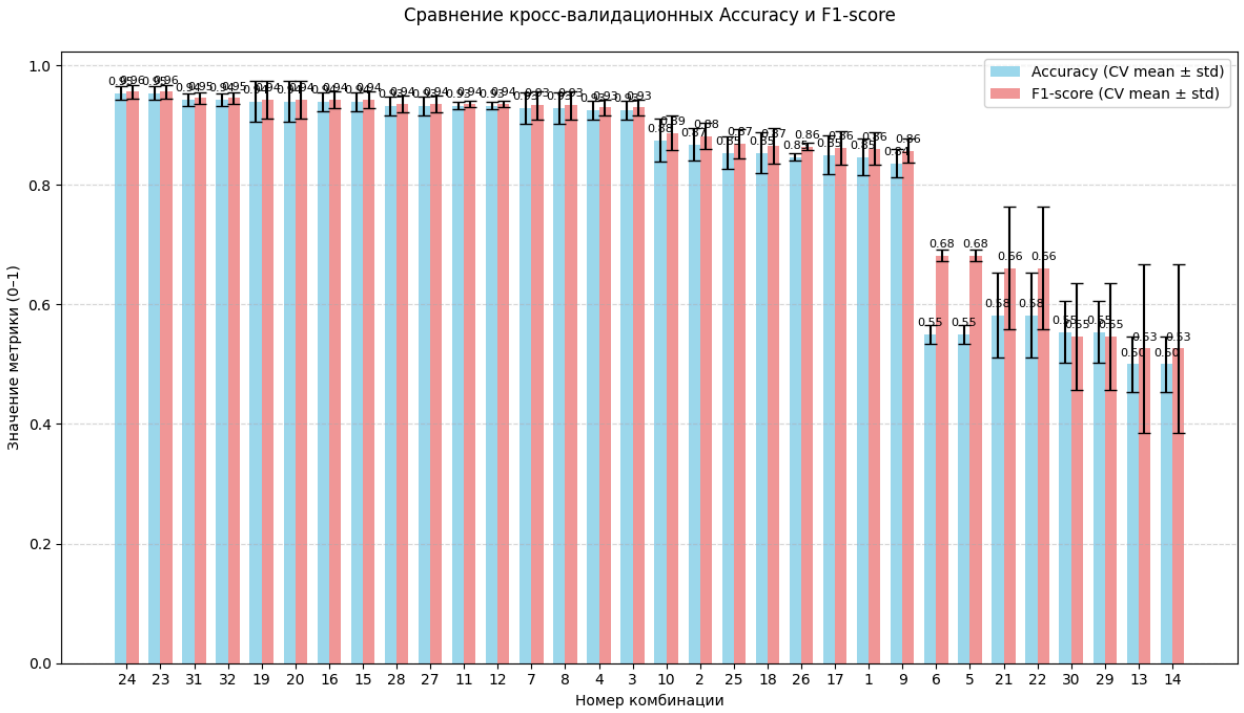


Рисунок 13 – Гистограмма с метриками для каждой комбинации гиперпараметров для трехслойной модели с балансировкой *SMOTE*

По результатам исследования можно сделать следующие выводы:

- Гипотеза полностью подтвердилась (все метрики выросли, причём значительно).

- Решение проблемы дисбаланса классов увеличила точность, *F*-меру и устойчивость моделей значительно. При этом стоит учесть, что данная модернизация позволила однослойной модели получить наилучшие показатели качества.

- Увеличение числа слоев не приводит к улучшению. Из этого можно сделать предположение, что однослойной модели достаточно для решения задачи.

- Лучшая пара на основе результатов обучения – ‘*relu*’ + ‘*lbfgs*’.

- Алгоритм ‘*adam*’ при применении балансировки полностью проигрывает ‘*lbfgs*’.

2.6 Проведение исследований с подбором гиперпараметров для данных с нормализацией параметров и балансировкой ADASYN

ADASYN (Adaptive Synthetic Sampling) [4] аналогичен SMOTE, но генерирует больше синтетических примеров в тех областях пространства признаков, где классификация целевого класса затруднена (вблизи границы решений). Это может помочь точнее описать сложные участки границы и дополнительно улучшить *F*-меру. Однако создание точек в пограничных зонах способно внести шум, поэтому эффективность метода нуждается в экспериментальной проверке.

Гипотеза – точность, *F*-мера и устойчивость моделей будут незначительно отличны от тех, что были получены с помощью *SMOTE*.

Варианты гиперпараметров для данных с учетом нормализации данных и балансировки *ADASYN* представлен в листинге 5.

Листинг 5 – Варианты гиперпараметров для данных с нормализацией и *ADASYN*

```
1.  # для одного слоя
2.  hidden_opts = [(50,), (100,), (150,), (200,)]
3.  act_opts = ['relu', 'tanh']
4.  solv_opts = ['adam', 'lbfgs']
5.  iter_opts = [500, 1000]
6.
7.  # для двух слоев
8.  hidden_opts = [(20, 10), (20, 20), (40, 30), (50, 20), (100, 50)]
9.  act_opts = ['relu', 'tanh']
10. solv_opts = ['adam', 'lbfgs']
11. iter_opts = [500, 1000]
12.
13. # для трех слоев
14. hidden_opts = [ (10, 10, 10), (30, 30, 20), (32, 32, 16), (48, 48, 32), (64, 64, 32)]
15. act_opts = ['relu', 'tanh']
16. solv_opts = ['adam', 'lbfgs']
17. iter_opts = [100, 1000]
```

После проведения экспериментов были получены следующие результаты:

- Лучший результат модели с одним слоем: *hidden_layer_sizes*=(200,), *activation*='*relu*', *solver*='*lbfgs*', *max_iter*=1000. ***Accuracy = 0.9522 ± 0.0191, F1-score = 0.9532 ± 0.0176.***

- Лучший результат модели с двумя слоями: *hidden_layer_sizes*=(40, 30), activation='relu', solver='lbfgs', max_iter=1000. ***Accuracy = 0.9412 ± 0.0104, F1-score = 0.9418 ± 0.0091.***

- Лучший результат модели с тремя слоями: *hidden_layer_sizes*=(48, 48, 32), *activation*='*relu*', *solver*='*lbfgs*', *max_iter*=1000. ***Accuracy = 0.9265 ± 0.0104, F1-score = 0.9286 ± 0.0101.***

На данном этапе лучший результат показала **однослойная нейронная сеть** (см. рис. 14-15).

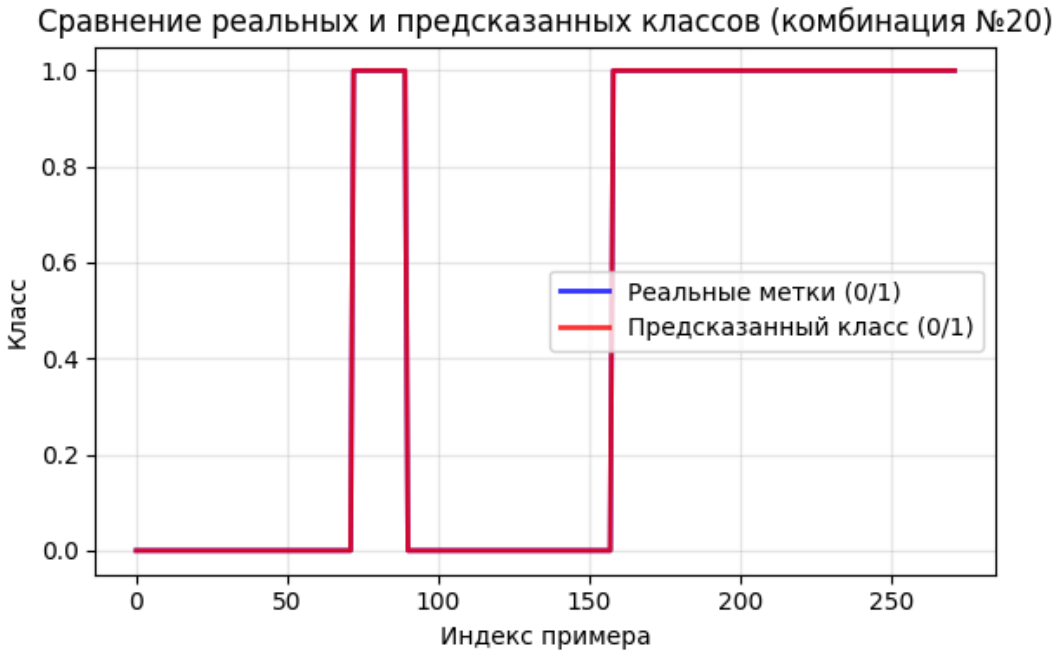


Рисунок 14 – График сравнения табличного значения выходной величины и выхода многосвязной сети для трехслойной модели с балансировкой *ADASYN*

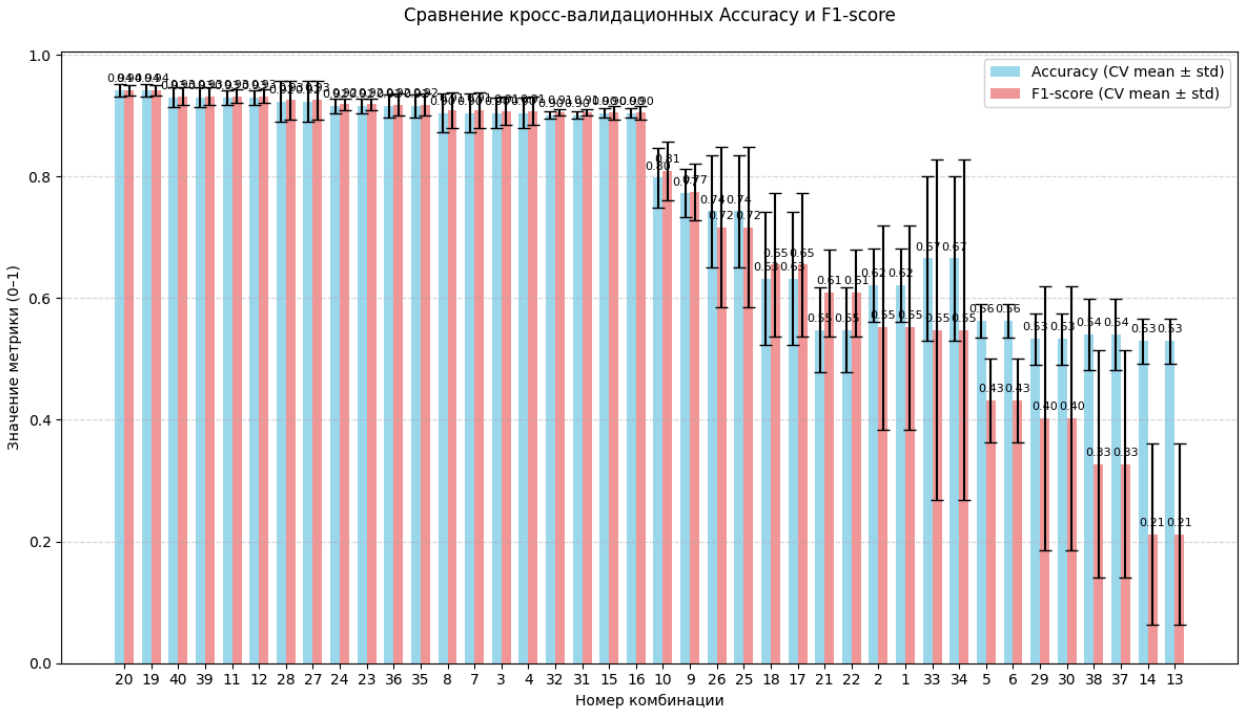


Рисунок 15 – Гистограмма с метриками для каждой комбинации гиперпараметров для трехслойной модели с балансировкой *ADASYN*

По результатам исследования можно сделать следующие выводы:

- Гипотеза полностью подтвердились. Однако явной закономерности в изменении не наблюдается (показатели однослойной модели снизились, но у других моделей повысились).
- Стандартные отклонения у *ADASYN* выше, что может свидетельствовать о возможном внесении шума в пограничных областях.
- Алгоритм ‘*adam*’ во всех трёх моделях значительно проигрывает ‘*lbfgs*’.
- Увеличение числа слоев не приводит к улучшению. Из этого можно сделать предположение, что однослойной модели достаточно для решения задачи.
- Лучшая пара на основе результатов обучения – ‘*relu*’ + ‘*lbfgs*’.
- Алгоритм ‘*adam*’ при применении балансировки полностью проигрывает ‘*lbfgs*’.

2.7 Проведение исследований с использованием Optuna

На предыдущих этапах перебор гиперпараметров выполнялся по фиксированным значениям. Это даёт возможность исследовать все комбинации выбранных гиперпараметров, но не позволяет исследовать промежуточные значения, которые могут давать результат лучше.

Тем более, если перебирать большое число гиперпараметров, обучение может длиться неделями. Поэтому было принято решение о внедрении Optuna. Это библиотека автоматической оптимизации гиперпараметров на основе байесовского подхода. В основе Optuna лежит алгоритм TPE (Tree-structured Parzen Estimator), который итеративно строит вероятностную модель связи гиперпараметров с целевой метрикой и предлагает новые комбинации, всё более точно приближаясь к оптимуму.

Данный подход может позволить найти лучшую конфигурацию за меньшее число итераций. Однако, Optuna не гарантирует лучшее решение, так как также может застревать в локальном минимуме.

По сравнению с предыдущими разделами в пространство поиска добавлен гиперпараметр ‘*alpha*‘ (коэффициент *L2*-регуляризации). Он штрафует большие веса модели, что может помочь предотвратить переобучение.

Гипотеза – точность и *F*-мера моделей будут лучше на всех моделях, но незначительно.

Варианты гиперпараметров для данных с применением *Optuna* представлены в листинге 6. Данные диапазоны и варианты гиперпараметров аналогичны для балансировки *SMOTE* и *ADASYN*.

Листинг 6 – Варианты гиперпараметров для данных с применением *Optuna*

```
1. # для одного слоя
2. n_trials = 150
3. n_neurons = trial.suggest_int('n_neurons', 10, 150)
4. activation = trial.suggest_categorical('activation', ['relu', 'tanh'])
5. solver = trial.suggest_categorical('solver', ['adam', 'lbfgs'])
6. max_iter = trial.suggest_int('max_iter', 10, 1000, step=10)
7. alpha = trial.suggest_float('alpha', 1e-4, 0.1, log=True)
```

```
8.
9.  # для двух слоев
10. n_trials = 150
11. n1 = trial.suggest_int('n_neurons_1', 30, 80)
12. n2 = trial.suggest_int('n_neurons_2', 10, 60)
13. activation = trial.suggest_categorical('activation', ['relu', 'tanh'])
14. solver = trial.suggest_categorical('solver', ['adam', 'lbfgs'])
15. max_iter = trial.suggest_int('max_iter', 10, 1000, step=10)
16. alpha = trial.suggest_float('alpha', 1e-4, 0.1, log=True)
17.
18. # для трех слоев
19. n_trials = 250
20. n1 = trial.suggest_int('n_neurons_1', 30, 70)
21. n2 = trial.suggest_int('n_neurons_2', 30, 70)
22. n3 = trial.suggest_int('n_neurons_3', 20, 70)
23. activation = trial.suggest_categorical('activation', ['relu', 'tanh'])
24. solver = trial.suggest_categorical('solver', ['lbfgs', 'adam'])
25. max_iter = trial.suggest_int('max_iter', 10, 1000, step=10)
26. alpha = trial.suggest_float('alpha', 1e-4, 0.1, log=True)
```

Работа *Optuna* направлена на максимальное значение *F*-меры модели. Процесс работы *Optuna* представлен на рисунке 16.

```
...
[I 2026-05-01 21:13:19,573] Trial 246 finished with value: 0.9303461356092936
[I 2026-05-01 21:13:21,097] Trial 247 finished with value: 0.9276814929843047
[I 2026-05-01 21:13:23,388] Trial 248 finished with value: 0.9402631578947368
[I 2026-05-01 21:13:25,455] Trial 249 finished with value: 0.9368185744898073
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output setting
=== Трёхслойная сеть ===
Лучший F1 (CV): 0.9592642613672905
Лучшие параметры: {'n_neurons_1': 30, 'n_neurons_2': 51, 'n_neurons_3': 49, '
```

Рисунок 16 – Процесс работы *Optuna* для трехслойной модели со *SMOTE*

После проведения экспериментов были получены следующие результаты:

- Балансировка *SMOTE*
  - Лучший результат модели с одним слоем: *activation*='*relu*', *solver*='*lbfgs*', *max_iter*=470, *alpha*=0.000616977786, *hidden_layer_sizes*=(63,). ***Accuracy = 0.9607 ± 0.0234, F1-score = 0.9627 ± 0.0222.***

○ Лучший результат модели с двумя слоями: *activation='relu'*, *solver='lbfgs'*, *max_iter*=600, *alpha*=0.00359876559, *hidden_layer_sizes*=(30, 19). ***Accuracy = 0.9536 ± 0.0186, F1-score = 0.9559 ± 0.0173.***

○ Лучший результат модели с тремя слоями: *activation='relu'*, *solver='lbfgs'*, *max_iter*=670, *alpha*=0.01269144843, *hidden_layer_sizes*=(30, 51, 49). ***Accuracy = 0.9571 ± 0.0202, F1-score = 0.9593 ± 0.0186.***

- Балансировка *ADASYN*

○ Лучший результат модели с одним слоем: *activation='relu'*, *solver='lbfgs'*, *max_iter*=230, *alpha*=0.0032129369905, *hidden_layer_sizes*=(66,). ***Accuracy = 0.9596 ± 0.0241, F1-score = 0.9606 ± 0.0234.***

○ Лучший результат модели с двумя слоями: *activation='relu'*, *solver='lbfgs'*, *max_iter*=190, *alpha*=0.00359876559, *hidden_layer_sizes*=(57, 30). ***Accuracy = 0.9485 ± 0.0164, F1-score = 0.9499 ± 0.0153.***

○ Лучший результат модели с тремя слоями: *activation='tanh'*, *solver='lbfgs'*, *max_iter*=540, *alpha*=0.0013197827316, *hidden_layer_sizes*=(60, 60, 56). ***Accuracy = 0.9559 ± 0.0180, F1-score = 0.9568 ± 0.0167.***

На данном этапе лучший результат показала **трехслойная нейронная сеть с балансировкой *SMOTE*** (см. рис. 17).

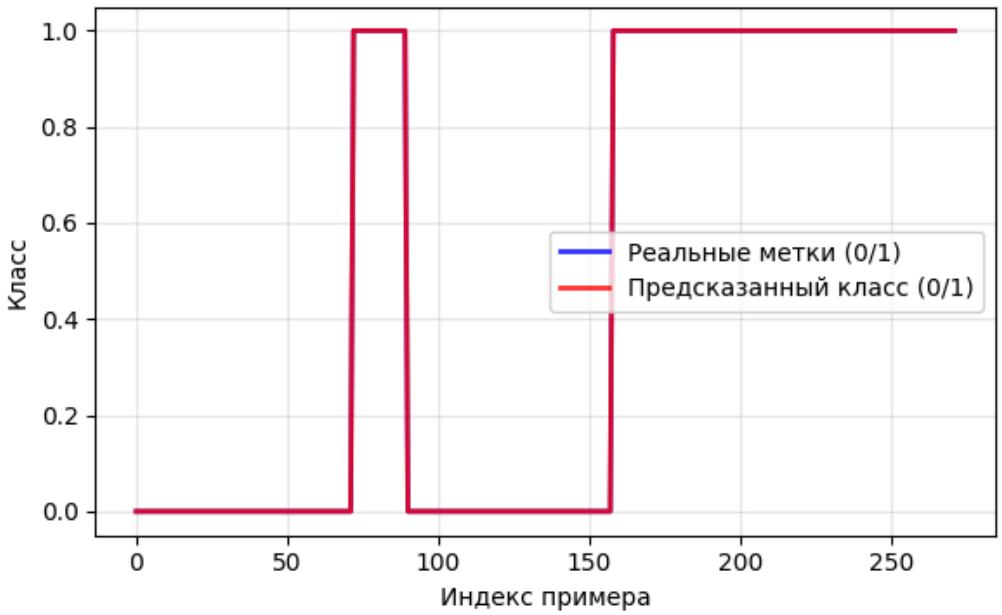


Рисунок 17 – График сравнения табличного значения выходной величины и выхода трехслойной модели с балансировкой *SMOTE*

По результатам исследования можно сделать следующие выводы:

- Гипотеза полностью подтвердились, точность и *F*-мера незначительно выросли, при этом упала устойчивость моделей.
- Практически всегда Optuna приходила к связке ‘*relu*’ + ‘*lbfgs*’.
- Во всех моделях гиперпараметр ‘*alpha*’ сильно отличается от значения по умолчанию.
- Значения *max_iter* во всех моделях меньше 700. Это отличается от тех предположений, которые были сделаны при выполнении обучения перебором гиперпараметров.
- Однослойные модели продолжают лидировать по точности и F-мере, однако у них также страдает устойчивость.
- Разница между *SMOTE* и *ADASYN* минимальна.

2.8 Подведение итогов исследований

По результатам всех этапов была сформирована таблица 1.

Таблица 1 – Результаты исследований *MLPClassifier* на выборке *AB*

№	Обработка данных	Архитектура скрытых слоёв	Алгоритм оптимизации	Функция активации	Максимальное число итераций	Коэффициент L2-регуляриза-ции	Метрики со стандартным отклонением	
							Accuracy ± std	F1-score ± std
1	Без дополнительной обработки	(50,)	lbfgs	relu	1000	0.0001	0.8182 ± 0.0000	0.5250 ± 0.0433
2		(50, 20)	lbfgs	relu	1000	0.0001	0.8295 ± 0.0254	0.6055 ± 0.0134
3		(32, 32, 16)	adam	relu	1000	0.0001	0.8523 ± 0.0254	0.5917 ± 0.0759
4	Сортировка относительно целевого класса	(50,)	lbfgs	relu	1000	0.0001	0.8182 ± 0.0000	0.5250 ± 0.0433
5		(50, 20)	adam	relu	1000	0.0001	0.8409 ± 0.0161	0.5893 ± 0.0246
6		(64, 64, 32)	lbfgs	relu	1000	0.0001	0.8409 ± 0.0161	0.6213 ± 0.0406
7	С нормализацией *MinMaxScaler*	(50,)	lbfgs	tanh	500	0.0001	0.8409 ± 0.0161	0.6307 ± 0.0280
8		(50, 20)	adam	relu	1000	0.0001	0.8466 ± 0.0188	0.6412 ± 0.0278
9		(30, 30, 20)	adam	relu	1000	0.0001	0.8693 ± 0.0188	0.6671 ± 0.0389
10	С нормализацией *MinMaxScaler* и балансировкой *SMOTE*	**(150,)**	**lbfgs**	**tanh**	**1000**	**0.0001**	**0.9536 ± 0.0118**	**0.9558 ± 0.0109**
11		(40, 30)	lbfgs	relu	1000	0.0001	0.9357 ± 0.0071	0.9370 ± 0.0075
12		(48, 48, 32)	lbfgs	relu	1000	0.0001	0.9393 ± 0.0186	0.9426 ± 0.0171
13	С нормализацией	**(200,)**	**lbfgs**	**relu**	**1000**	0.0001	**0.9522 ± 0.0191**	**0.9532 ± 0.0176**

14	*MinMaxScaler* и балансировкой *ADASYN*	(40, 30)	lbfgs	relu	1000	0.0001	0.9412 ± 0.0104	0.9418 ± 0.0091
15		(48, 48, 32)	lbfgs	relu	1000	0.0001	0.9265 ± 0.0104	0.9286 ± 0.0101
16	С нормализацией *MinMaxScaler* и балансировкой *SMOTE* + *Optuna*	**(63,)**	**lbfgs**	**relu**	**470**	**0.0006169778**	**0.9607 ± 0.0234**	**0.9627 ± 0.0222**
17		(30, 19)	lbfgs	relu	600	0.0035987656	0.9536 ± 0.0186	0.9559 ± 0.0173
18		**(30, 51, 49)**	**lbfgs**	**relu**	**670**	**0.0126914484**	**0.9571 ± 0.0202**	**0.9593 ± 0.0186**
19	С нормализацией *MinMaxScaler* и балансировкой *ADASYN* + *Optuna*	**(66,)**	**lbfgs**	**relu**	**230**	**0.0032129369**	**0.9596 ± 0.0241**	**0.9606 ± 0.0234**
20		(57, 30)	lbfgs	relu	190	0.0320593885	0.9485 ± 0.0164	0.9499 ± 0.0153
21		**(60, 60, 56)**	**lbfgs**	**tanh**	**540**	**0.0013197827**	**0.9559 ± 0.0180**	**0.9568 ± 0.0167**

По представленным результатам можно сделать следующие выводы:

- Без предобработки модели демонстрируют низкую *F*-меру (0.52–0.62) и нестабильность.
- Нормализация улучшает метрики, но ключевой прорыв достигается только после балансировки классов *SMOTE* и *ADASYN*. *SMOTE* и *ADASYN* показали практически идентичные результаты
- Эффект от применения *Optuna* практически незаметен, даже с учётом добавления коэффициента *L2*-регуляризации.
- При достаточной обработке данных точность и *F*-мера односвязной сети выходит в лидеры, однако в большинстве она проигрывает в устойчивости.
- Практически во всех моделях после проведения нормализации лучшей связкой является ‘*relu*’ + ‘*lbfgs*’.

Для проверки всего этапа проведения исследований проведем проверку данных гиперпараметров на выборке *C*.

3 Тестирование модели на тестовой выборке *C*

3.1 Загрузка и обработка новой выборки данных

Способ обработки выборки *C* идентичен обработке выборки *AB*.

По итогам анализа были сделаны следующие выводы:

1. Отсутствие *NaN*-ячеек данных и дубликатов (все данные уникальны).
2. Датасет содержит 58 строк с информацией о данных датчиков для 5 типов поверхностей.
3. В датасете наблюдается дисбаланс классов при условии идентификации одного типа поверхности.
4. В датасете отсутствуют выбросы.
5. В приложении Б представлены графики каждого из столбцов. Стоит отметить, что динамика графиков отличается от динамики графиков в исходной выборке.
6. Матрицы корреляции (см. рис. 18) имеют примерно идентичное распределение, однако само значение корреляции параметров в среднем снизилось. Это может свидетельствовать о том, что данные были сняты при идентичных условиях, что и в исходном датасете.

	I1	I2	I3	gx	gy	gz	ax	ay	az	V1real	V2real	V3real	Tx	Ty	Tφ	Tz	Type
I1	100%	43%	83%	12%	18%	25%	3%	2%	8%	30%	13%	21%	4%	36%	34%	27%	13%
I2	43%	100%	54%	2%	7%	6%	1%	0%	10%	14%	17%	5%	4%	4%	21%	10%	13%
I3	83%	54%	100%	1%	25%	30%	21%	5%	8%	35%	26%	8%	16%	21%	42%	18%	3%
gx	12%	2%	1%	100%	33%	24%	1%	8%	23%	23%	33%	9%	14%	22%	22%	22%	10%
gy	18%	7%	25%	33%	100%	95%	2%	3%	33%	88%	91%	75%	40%	23%	91%	74%	1%
gz	25%	6%	30%	24%	95%	100%	5%	1%	36%	93%	94%	79%	37%	24%	94%	76%	1%
ax	3%	1%	21%	1%	2%	5%	100%	22%	19%	10%	4%	7%	14%	3%	3%	2%	20%
ay	2%	0%	5%	8%	3%	1%	22%	100%	13%	2%	2%	5%	4%	11%	0%	5%	10%
az	8%	10%	8%	23%	33%	36%	19%	13%	100%	30%	32%	26%	9%	8%	34%	39%	6%
V1real	30%	14%	35%	23%	88%	93%	10%	2%	30%	100%	82%	64%	42%	15%	86%	66%	0%
V2real	13%	17%	26%	33%	91%	94%	4%	2%	32%	82%	100%	64%	42%	44%	90%	69%	1%
V3real	21%	5%	8%	9%	75%	79%	7%	5%	26%	64%	64%	100%	6%	7%	69%	66%	0%
Tx	4%	4%	16%	14%	40%	37%	14%	4%	9%	42%	42%	6%	100%	33%	34%	23%	3%
Ty	36%	4%	21%	22%	23%	24%	3%	11%	8%	15%	44%	7%	33%	100%	20%	15%	7%
Tφ	34%	21%	42%	22%	91%	94%	3%	0%	34%	86%	90%	69%	34%	20%	100%	81%	1%
Tz	27%	10%	18%	22%	74%	76%	2%	5%	39%	66%	69%	66%	23%	15%	81%	100%	1%
Type	13%	13%	3%	10%	1%	1%	20%	10%	6%	0%	1%	0%	3%	7%	1%	1%	100%

Рисунок 18 – Матрица корреляции для выборки *C*

3.2 Оценка качества ранее найденных моделей

По результатам всех этапов была сформирована таблица 2.

Таблица 2 – Результаты исследований MLPClassifier для тестовой выборки *C*

№	Обработка данных	Архитектура скрытых слоёв	Алгоритм оптимизации	Функция активации	Максимальное число итераций	Коэффициент L2-регуляризации	Метрики	
							Accuracy	F1-score
1	С нормализацией *MinMaxScaler*	**(50,)**	**lbfgs**	**tanh**	**500**	**0.0001**	**0.8793**	**0.7407**
2		(50, 20)	adam	relu	1000	0.0001	0.8966	0.7000
3		**(30, 30, 20)**	**adam**	**relu**	**1000**	**0.0001**	**0.9138**	**0.7368**
4	С нормализацией *MinMaxScaler* и балансировкой *SMOTE*	(150,)	lbfgs	tanh	1000	0.0001	0.8621	0.6000
5		(40, 30)	lbfgs	relu	1000	0.0001	0.8448	0.6667
6		(48, 48, 32)	lbfgs	relu	1000	0.0001	0.8448	0.6400
7	С нормализацией *MinMaxScaler* и балансировкой *ADASYN*	**(200,)**	**lbfgs**	**relu**	**1000**	**0.0001**	**0.8793**	**0.7200**
8		(40, 30)	lbfgs	relu	1000	0.0001	0.8793	0.6316
9		(48, 48, 32)	lbfgs	relu	1000	0.0001	0.8621	0.6667
10	С нормализацией *MinMaxScaler* и балансировкой *SMOTE* + *Optuna*	(63,)	lbfgs	relu	470	0.0006169778	0.8793	0.6667
11		(30, 19)	lbfgs	relu	600	0.0035987656	0.8276	0.5455
12		(30, 51, 49)	lbfgs	relu	670	0.0126914484	0.8103	0.6452
13	С нормализацией *MinMaxScaler* и	(66,)	lbfgs	relu	230	0.0032129369	0.8621	0.6667

14	балансировкой *ADASYN* + *Optuna*	(57, 30)	lbfgs	relu	190	0.0320593885	0.8276	0.5833
15		(60, 60, 56)	lbfgs	tanh	540	0.0013197827	0.8793	0.6957
16	С нормализацией *MinMaxScaler* и *ADASYN* + *собственные значения*	**(30, 30, 20)**	**lbfgs**	**relu**	**1000**	**0.00016077399**	**0.9138**	**0.8148**

По представленным результатам можно сделать следующие выводы:

- При подборе собственных значений гиперпараметров было достигнуто значительно лучшее качество модели. Однако этот результат получен путём подстройки под конкретную выборку *C*, что иметь слабую обобщающую способность.

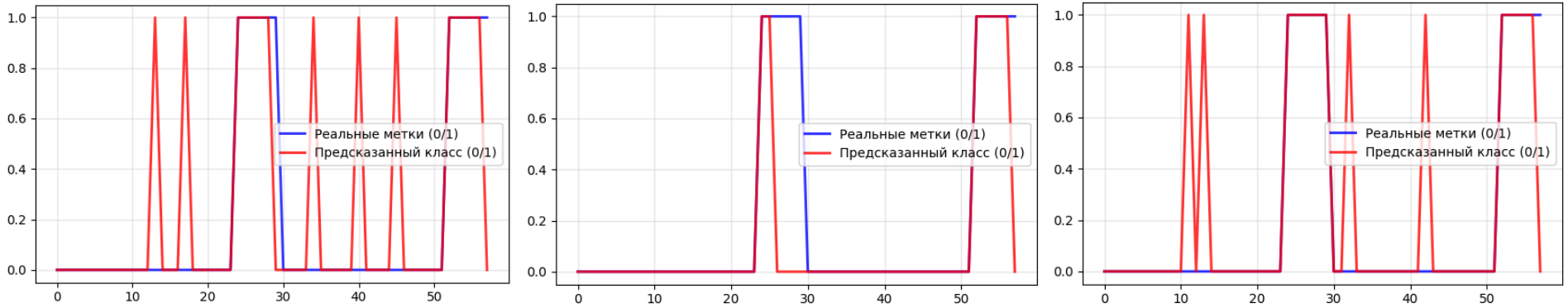
- Модели с *Optuna* на независимой выборке показали более низкое качество, чем конфигурации из простого перебора. Это указывает на переобучение даже при использовании *L2*-регуляризации. Скорее всего, Optuna подстроилась под модель *AB*.

- Самым неожиданным результатом стало то, что модели с нормализацией **без балансировки классов** оказались точнее и стабильнее на тестовой выборке, чем модели с *SMOTE* и *ADASYN*. Это может говорить о том, что *SMOTE* и *ADASYN* могли внести данные, которые не соответствуют тем признакам, которые находит модель. Однако малый размер выборки *C* не позволяет сделать вывод об обобщающей способности модели в целом, поэтому для окончательных выводов требуется большее количество тестовых данных.

- Если не учитывать собственные значения гиперпараметров, победила модель трехслойная сеть с нормализацией MinMaxScaler и без балансировки под номером 3. Однако, так как в выборке C мало данных, модель могла просто угадать. Для проверки гипотезы нужна достаточно большая выборка данных (от 300 строк).

- Лучшей парой гиперпараметров остались ‘*relu*’ + ‘*lbfgs*’. Стоит учесть, что в нормализации без балансировки данное соотношение практически не фигурирует и лучшим оказалось ‘*tanh*’ + ‘*lbfgs*’ и ‘*relu*’ + ‘*adam*’.

- В качестве основного кандидата для проверки на выборке D выбрана **трехслойная сеть с нормализацией данных без балансировки с гиперпараметрами: *activation*='*relu*', *solver*='*adam*', *max_iter*=1000, *hidden_layer_sizes*=(30, 30, 20) (см. рис. 19)**. Однако, если окажется, что данная модель имеет слабую обобщающую способность, то есть второй вариант модели – Нормализация данных без балансировки с гиперпараметрами: *activation*='*tanh*', *solver*='*lbfgs*', *max_iter*=500, *hidden_layer_sizes*=(50,).



а б в

а – выборка гиперпараметров под №1; б – выборка гиперпараметров под №3; в – выборка гиперпараметров под №16

Рисунок 18 – Матрица корреляции для выборки *C*

Вывод

В ходе выполнения лабораторной работы был построен нейросетевой классификатор MLPClassifier для идентификации типа подстилающей поверхности робототехнической системы, функционирующей в условиях физически неоднородной среды. Исходная выборка объемом 176 наблюдений характеризовалась значительным дисбалансом классов (целевой класс составлял около 20% всей выборки).

При обработке датасета было выявлено, что в данных отсутствовали дубликаты, NaN-ячейки и выбросы (с учётом работы датчиков). После добавления недостающих подмножеств был проведен повторный анализ, после которого было произведено кэпирование по перцентилям (для фильтрации выбросов) и построены графики всех данных (приложение А) и матрица корреляции.

После обработки данных было последовательно исследовано влияние различных способов предобработки данных и настройки гиперпараметров (методом перебора):

- Без обработки и с сортировкой относительно целевого класса модели показали низкие и нестабильные значения *F*-меры (0.52–0.62), при этом сортировка не дала значимого эффекта, так как применяемая стратифицированная кросс-валидация с перемешиванием сама обеспечивает равномерное распределение целевого класса в фолдах.
- Нормализация с *MinMaxScaler* несколько улучшила метрики (*F*-мера до 0.67) и повысила устойчивость, но показатели также остаются низкими.
- Балансировка классов методами *SMOTE* и *ADASYN* привела к качественному скачку всех метрик (*F*-мера достигла 0.93–0.96). Оба метода показали практически эквивалентные результаты на выборке *AB*.
- Оптимизация гиперпараметров с помощью *Optuna* позволила незначительно повысить *F*-меру. При этом при использовании *Optuna* был добавлен гиперпараметр ‘alpha’ (коэффициент *L2*-регуляризации), который в

теории должен уменьшить риск переобучения модели. Однако, устойчивость моделей снизилась, что имеет риск при проверке на тестовой выборке.

- Практически во всех вариантах с нормализацией доминировала пара ‘*relu*’ + ‘*lbfgs*’. Однослойные сети после балансировки не уступали двух- и трехслойным моделям по качеству.

Тестирование на отложенной выборке C показало неожиданные результаты:

● Наилучший результат дал ручной подбор гиперпараметров с нормализацией и балансировкой *ADASYN*, но такая настройка под конкретную выборку может привести к слабой обобщающей способности.

● Модели с *Optuna* уступили простому перебору, что указывает на переобучение даже при использовании *L2*-регуляризации.

● Неожиданно, модели с нормализацией без балансировки оказались точнее и стабильнее, чем модели с *SMOTE* и *ADASYN*. Это может объясняться тем, что новые примеры целевого класса плохо соответствуют реальному распределению признаков в выборке *C*.

● В качестве основного кандидата для проверки на выборке *D* выбрана **трехслойная модель с нормализацией без балансировки: *activation*='*relu*', *solver*='*adam*', *max_iter*=1000, *hidden_layer_sizes*=(30, 30, 20)**. В качестве запасного варианта выступила *однослойная модель с нормализацией без балансировки: activation='tanh', solver='lbfgs', max_iter=500, hidden_layer_sizes=(50,)*.

Поставленные цели лабораторной работы были достигнуты.

Список используемых источников

1. Модели нейронных сетей (контролируемые) – [Электронный ресурс] – scikit learn – Режим доступа: https://scikit-learn.ru/stable/modules/neural_networks_supervised.html (дата обращения: 30.05.2026).

2. Кросс-валидация (*Cross-validation* или перекрестная проверка): оценка эффективности оценок – [Электронный ресурс] – scikit learn – Режим доступа: https://scikit-learn.ru/stable/modules/cross_validation.html (дата обращения: 30.05.2026).

3. *SMOTE* – [Электронный ресурс] – imbalanced learn – Режим доступа: https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html (дата обращения: 30.05.2026).

4. *ADASYN* – [Электронный ресурс] – imbalanced learn – Режим доступа: https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.ADASYN.html (дата обращения: 30.05.2026).

Приложение А
(обязательное)
Графики параметров выборки *AB* датасета

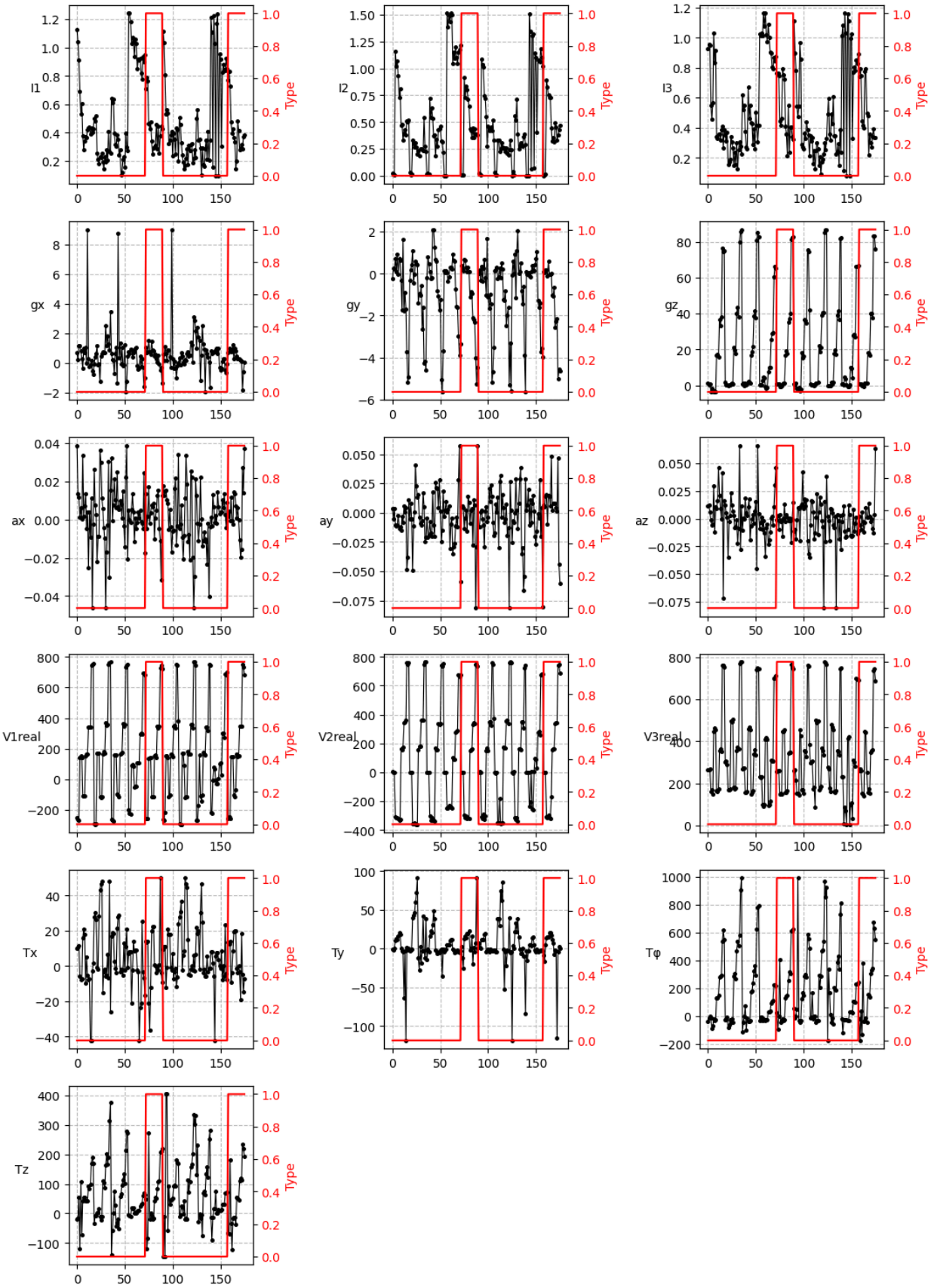


Рисунок А.1 – Графики данных выборки *AB*

Приложение Б
(обязательное)
Графики параметров выборки *C*

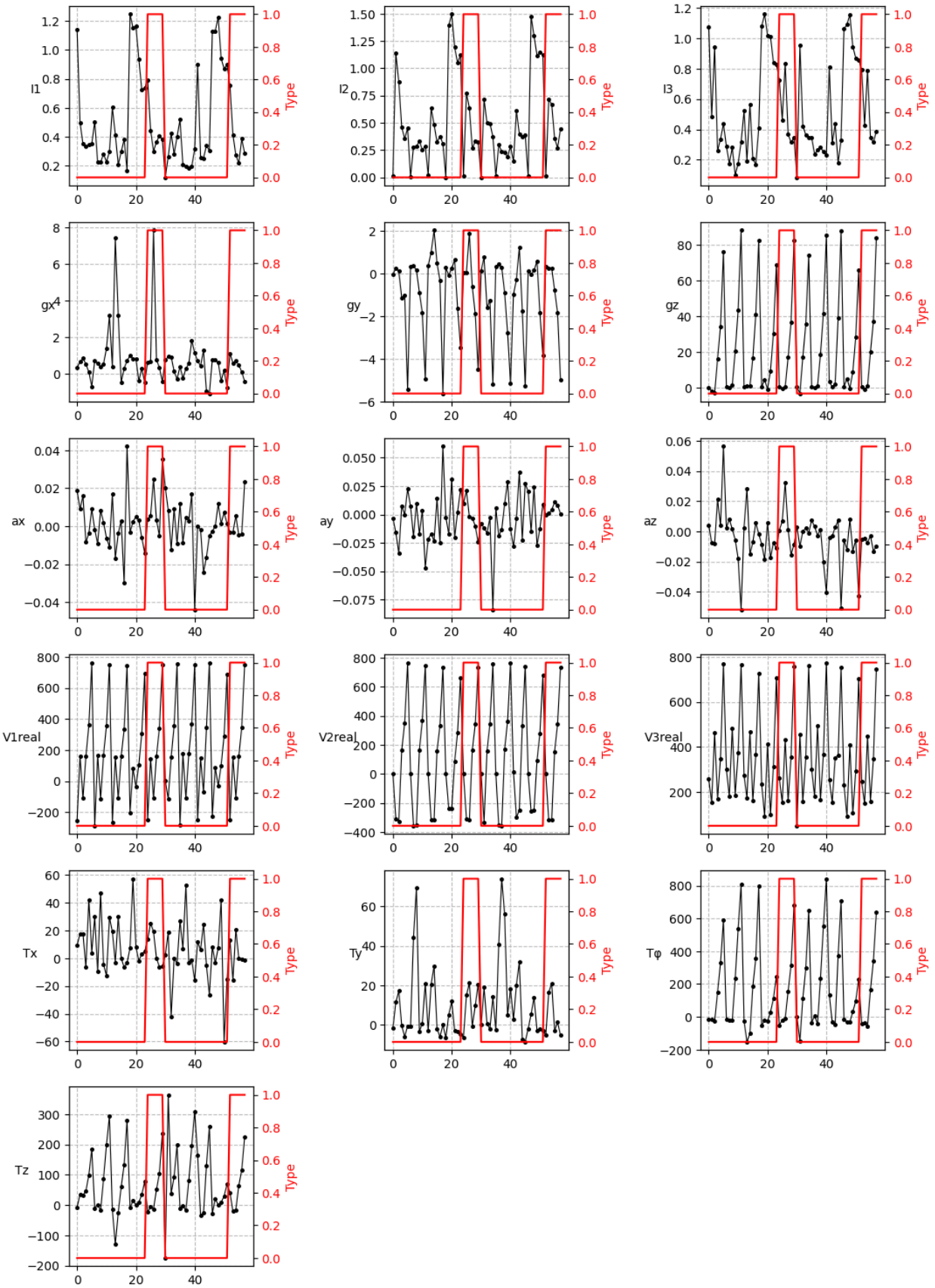


Рисунок Б.1 – Графики данных выборки *C*